SEGi UNIVERSITY

CHAPTER 1: INTRODUCTION TO ALGORITHMS & COMPUTATIONAL THINKING

TOPIC OVERVIEW

This chapter introduces the basic ideas of algorithms and computational thinking. Students learn what an algorithm is, how it is used to solve problems, and why algorithms are important in computer science and everyday applications.

Students will explore:

- What defines an algorithm
- Key features of a good algorithm
- Examples of algorithms in daily life and computing
- Deterministic vs. non-deterministic algorithms
- Basic problem-solving skills in computational thinking
- A simple introduction to time and space efficiency

By the end of this chapter, students will understand the purpose of algorithms, how to write them using pseudocode or flowcharts, and how to think logically when solving problems.

SEGi UNIVERSITY

CHAPTER 1: INTRODUCTION TO ALGORITHMS & COMPUTATIONAL THINKING

LEARNING OUTCOMES

By the end of this chapter, students will be able to:

1. **Define** what an algorithm is and describe its key characteristics.
2. **Explain** the importance of algorithms in problem-solving and computing.
3. **Differentiate** between deterministic and non-deterministic algorithms.
4. **Apply** basic computational thinking skills such as decomposition and pattern recognition.
5. **Represent** simple algorithms using pseudocode and flowcharts.
6. **Describe** the basic idea of time and space efficiency in evaluating algorithms.

CHAPTER 1: INTRODUCTION TO ALGORITHMS & COMPUTATIONAL THINKING

1.0 Introduction

In computer science, an **algorithm** is a precise set of steps or instructions designed to solve a specific problem or perform a computation. Algorithms are the backbone of all software systems, determining how efficiently and effectively a program can process data, solve problems, or perform tasks.

IMPORTANCE OF ALGORITHMS

- Algorithms help computers solve problems efficiently.
- They allow us to automate tasks, such as searching for information, recommending videos, or calculating routes.
- Good algorithms make programs faster, more reliable, and easier to maintain.



Understanding algorithms helps students think logically, solve problems, and design solutions that are correct and efficient. It also lays the foundation for more advanced topics such as algorithm analysis, data structures, searching, sorting, and design techniques in later chapters.

1.2 Properties of a Good Algorithm

A good algorithm is more than just a list of steps; it ensures that solutions are correct, efficient, and usable in practice. Here are the key characteristics with more explanation:

1 Clear and Unambiguous

- Each instruction must be precise, leaving no room for interpretation.
- Students can imagine following the steps like a recipe.
- Example:
  - Ambiguous: “Sort the numbers somehow.”
  - Clear: “Arrange the numbers from smallest to largest using bubble sort.”

2 Well-Defined Inputs and Outputs

- Every algorithm starts with input data and produces an output.
- It should define what input it expects and what result it generates.
- Example:
  - Input: 3 numbers → 5, 2, 8
  - Output: The average = 5

3 Finiteness (Termination)

- An algorithm must stop after a finite number of steps.
- Infinite loops are not allowed in a proper algorithm.
- Example:
  - A loop that sums numbers from 1 to 10 stops after adding the tenth number

4 Effectiveness (Feasibility)

- Steps must be simple enough to be executed manually or by a computer.
- The instructions must be realistic; an algorithm that requires solving an unsolvable problem is not effective.
- Example:
  - Find the largest prime number below a million by checking each number sequentially is feasible, but find the largest prime below infinity is not.

5 Efficiency

- Efficiency measures how much time and memory the algorithm uses.
- Example:
  - Searching a list of 100 items using a linear search involves checking each item one by one, which is slower, whereas a binary search is faster because it repeatedly divides the list in half to find the desired item.

6 Generality (Applicability)

- The algorithm should work for a wide range of inputs, not just a specific case.
- Example:
  - An algorithm that adds any two numbers works for all numbers, not just 2 + 3.

7 Correctness

- The algorithm must produce the correct output for all valid inputs.
- An efficient algorithm that gives wrong answers is useless.

1.3 Deterministic vs Non-Deterministic Algorithms

Algorithms can be categorized based on how predictable their outcomes are. Understanding this helps students recognize the type of problems they are solving and how algorithms behave under different conditions.

1.3.1 Deterministic Algorithms

Deterministic algorithms produce exactly the same output every time for the same input. Their steps are fully defined and predictable, which makes them easy to understand, test, and debug. Deterministic algorithms are commonly used in most basic programming and problem-solving exercises.

EXAMPLE

1. Adding two numbers with a deterministic algorithm always produces the same result; if the inputs are 3 and 5, the output will always be 8.
2. Sorting a list such as [4, 1, 3] using a specific sorting method will consistently produce [1, 3, 4].





- Predictable and reliable.
- Easy to verify correctness.
- Ideal for teaching beginners, as results are consistent.

1.3.2 Non-Deterministic Algorithms

Non-deterministic algorithms may produce different outputs for the same input because they involve randomness, multiple possible paths, or probabilistic choices. These algorithms are used when a problem has many possible solutions or when exploring all possibilities deterministically is too slow or impractical.

EXAMPLE

1. Random number selection:
   Choosing a number randomly from 1 to 10 → result may vary each time.
2. Game AI decision-making:
   An AI may select different moves in the same game state t make gameplay unpredictable.
3. Randomized optimization algorithms:
   Algorithms that explore multiple solutions to find the best or approximate solution.



Non-deterministic algorithms are more flexible because they allow multiple possible outcomes, but this also makes them harder to predict and analyze compared to deterministic algorithms. They are commonly used in fields such as artificial intelligence, optimization, and simulations, where exploring different possibilities is beneficial.

1.4 Computational Thinking

Computational thinking is a problem-solving approach used in computer science to break down complex problems into simpler, manageable steps. It helps students think logically, identify patterns, and design structured solutions that can be executed by a computer. Even before writing code, computational thinking guides how we analyze a problem and plan an effective algorithm.

To gain a deeper understanding, let examine the topic through these key aspects: decomposition, pattern recognition, algorithms, and abstraction.

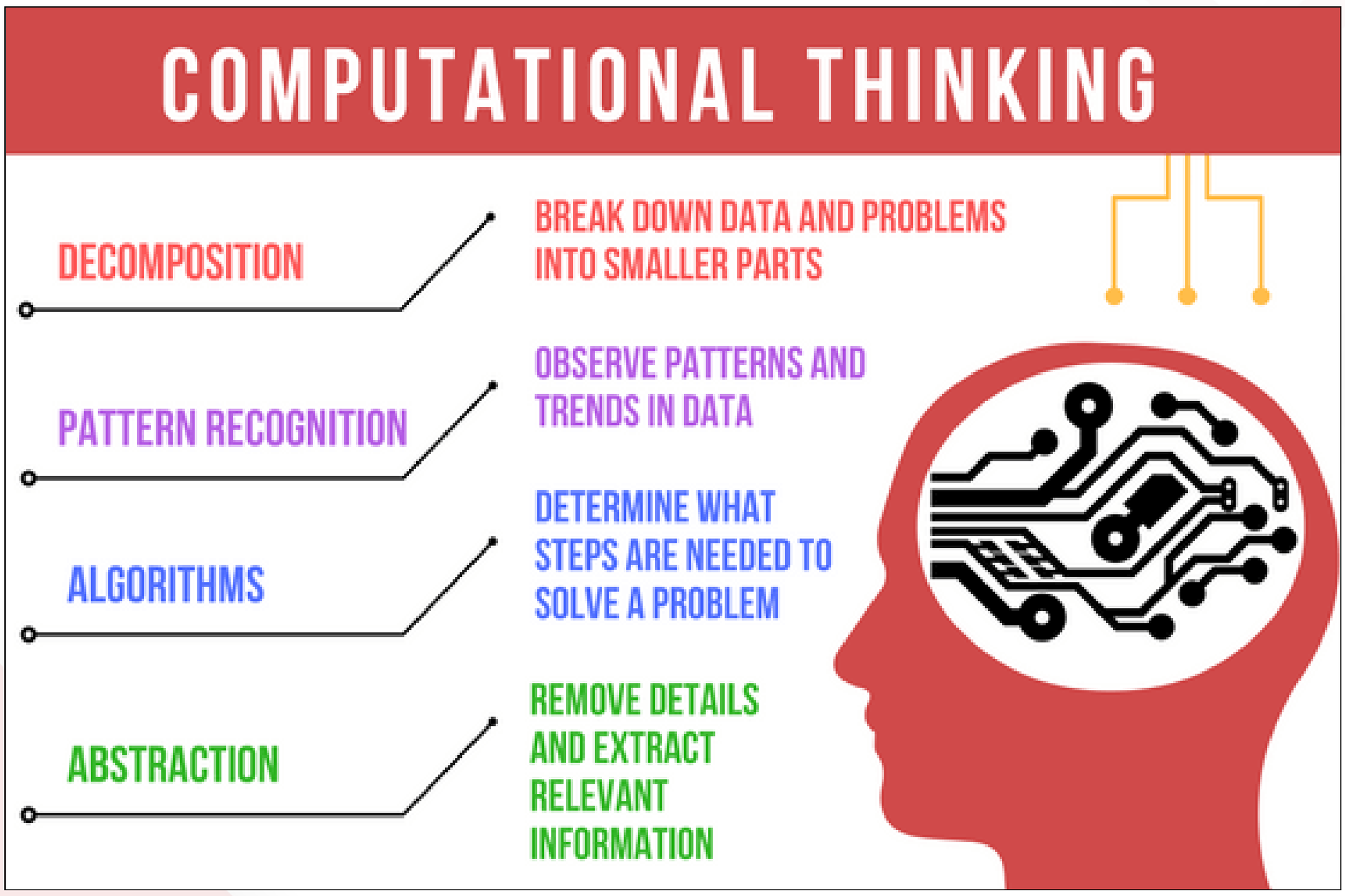


Figure 1.1 Embedding Computational Thinking

1.4.1 Decomposition

Decomposition is the process of breaking a large or complicated problem into smaller parts. Each smaller part is easier to understand and solve.

EXAMPLE

To create a food delivery system, we break the problem into parts such as user login, browsing menus, placing an order, making payment, and tracking delivery. Solving each component individually makes the entire system easier to develop



1.4.2 Pattern Recognition

Pattern recognition involves identifying similarities or repeated structures in problems. When we notice patterns, we can design solutions more quickly and avoid repeating work.

EXAMPLE

If you observe that sorting a list of names, numbers, or dates follows a similar process, you can reuse the same sorting logic for different data types.



1.4.3 Algorithmic Thinking

Algorithmic thinking is the ability to create a clear, step-by-step process to solve a problem. It involves arranging tasks logically so that anyone—or any program—can follow the instructions easily.

EXAMPLE

To withdraw cash from an ATM, you follow steps such as inserting the card, entering the PIN, choosing an amount, and collecting the money. Each step must occur in the right order for the process to work.



1.4.4 Abstraction

Abstraction means removing unnecessary details and focusing only on what is important to solve the problem.

EXAMPLE

When designing a game character, you focus on essential attributes like health, speed, and abilities, while ignoring irrelevant details such as the exact number of polygons used to draw the character.



Computational thinking are essential in designing algorithms, writing code, and solving real problems efficiently. It helps students think systematically, improves analytical skills, and prepares them for more advanced topics.

1.5 Representing Algorithms: Pseudocode and Flowcharts

There are several ways to express an algorithm clearly so that it can be understood, analyzed, and later converted into a program. To understand how algorithms can be represented, we can divide this topic into two main forms: pseudocode and flowcharts.

1.5.1 Pseudocode

Pseudocode is a simple, structured, and human-readable way to describe an algorithm. It is not written in any specific programming language, but it uses a logical format that resembles real code. Pseudocode helps students plan and organize their ideas before turning them into actual programs.

Characteristics of good pseudocode:

- Easy to read and understand
- Uses common keywords
- Shows the logical flow clearly
- Does not depend on syntax from any specific programming language

```
BEGIN
    INPUT A, B
    IF A > B THEN
        PRINT A
    ELSE
        PRINT B
    ENDIF
END
```

Pseudocode is useful because it allows us to plan a solution before writing real code, which helps reduce errors later. It also makes algorithms easier to explain and share with others since it focuses on the logic rather than the programming language. By using pseudocode, we can think more clearly about each step needed to solve a problem without worrying about syntax.

1.5.2 Flowcharts

A flowchart is a visual diagram that uses symbols and arrows to show the steps of an algorithm. It allows students to see how data moves and how decisions are made within a process.

Common flowchart symbols:

Symbol	Name	Function
	Start/end	An oval represents a start or end point
	Arrows	A line is a connector that shows relationships between the representative shapes
	Input/Output	A parallelogram represents input or output
	Process	A rectangle represents a process
	Decision	A diamond indicates a decision

Table 1.1 Flowchart Symbols

Flowcharts are useful because they provide a clear visual overview of how an algorithm works. By showing each step in a diagram, it becomes easier to spot errors, unnecessary loops, or missing processes. Flowcharts are also helpful when explaining logic to people who may not understand programming, as the visuals make the sequence of actions easy to follow. In addition, they play an important role in system design and documentation by offering a simple and organized representation of the workflow.

1.6 Introduction to Time and Space Efficiency

When designing algorithms, it is important to consider how efficiently they use resources. Two main resources are:

- time (how long an algorithm takes to run)
- space (how much memory or storage it uses)

- **Time efficiency** refers to how quickly an algorithm produces the correct output.
- Example:
  - Searching a list of 100 items using a linear search is slower because it checks each item one by one, whereas binary search is faster because it divides the list in half repeatedly.

- **Space efficiency** refers to how much memory the algorithm requires while solving the problem.
- Example:
  - storing intermediate results in a calculation may use more memory but can save time by avoiding repeated calculations.

At this introductory level, students only need an intuitive understanding: faster algorithms are better, and algorithms that use less memory are more efficient. These concepts prepare students for deeper analysis in later chapters.

1.7 Summary

This chapter introduced the fundamental concepts of algorithms and computational thinking. It covered the definition of an algorithm and its key characteristics, such as clarity, correctness, efficiency, and generality.

The difference between deterministic and non-deterministic algorithms was explained, highlighting how some algorithms always produce the same output while others may vary. The chapter also explored computational thinking skills, including decomposition, pattern recognition, abstraction, and algorithmic thinking, and demonstrated how to represent algorithms using pseudocode and flowcharts.

Finally, the basic idea of time and space efficiency was presented, illustrating how algorithms can differ in speed and memory usage. Understanding these concepts provides a strong foundation for planning solutions, thinking logically, and communicating algorithms effectively.



EXERCISES

1. Define an algorithm and explain its key characteristics.
2. Why are algorithms important in solving problems and computing tasks?
3. Differentiate between deterministic and non-deterministic algorithms with examples.
4. Explain the four main computational thinking skills: decomposition, pattern recognition, abstraction, and algorithmic thinking, with examples.
5. Write pseudocode for an algorithm that finds the largest of three numbers.
6. Draw a flowchart for an algorithm that checks whether a number is even or odd.
7. Explain the basic idea of time efficiency and space efficiency in evaluating algorithms.
8. Give one example of an algorithm in everyday life and explain why it is considered an algorithm.



REFLECTIVE QUESTIONS

1. How do algorithms affect everyday life beyond computers?
2. Why is planning an algorithm before coding important?
3. Can a more efficient algorithm always be better than a simpler one? Why or why not?